

실시간 운영체제 기반 전자제어기 프로그래밍

Contents

실시간 운영체제 기반 전자제어기
프로그래밍

Queue 관리

Queue Management

실시간 운영체제

□ Queue의 특성

○ 데이터 저장(*Data Storage*)

- Queue는 고정 크기의 데이터 항목을 유한 개수만큼 저장한다.
 - Queue가 동시에 보관할 수 있는 최대 항목 수를 Length라고 하며, Length와 각 항목의 크기는 Queue 생성 시 결정한다.
 - Queue는 일반적으로 FIFO(First In First Out) 버퍼로 사용한다.
 - 데이터는 Queue의 뒤쪽(Tail)에 쓰고 앞쪽(Head)에서 꺼낸다.
 - 필요하면 Queue의 앞쪽에 쓰거나, 앞쪽에 이미 존재하는 데이터를 덮어쓸 수도 있다.

➤ Queue에 데이터를 쓰고 읽는 순서

단계	상태/의미
초기	Queue는 비어 있다.
Write A	A를 Tail에 저장한다.
Write B	B를 A 뒤에 저장한다.
Read	Head의 A를 꺼내면 B가 다음 Head가 된다.
추가 연산	같은 방식으로 FIFO 순서를 유지한다.

실시간 운영체제

□ Queue의 특성

○ 데이터 저장(*Data Storage*)

- Queue 동작은 개념적으로 Queue by Copy와 Queue by Reference의 두 방식으로 구현할 수 있다.

방식	의미	특징
Queue by Copy	전송하는 데이터를 바이트 단위로 Queue 내부 저장 영역에 복사한다.	FreeRTOS가 기본적으로 사용하는 방식이다.
Queue by Reference	실제 데이터 대신 데이터를 가리키는 Pointer만 Queue에 저장한다.	큰 데이터를 효율적으로 전달할 수 있지만 메모리 소유권과 유효성을 직접 관리해야 한다.

- FreeRTOS가 Queue by Copy를 채택한 이유는 Queue by Reference보다 단순하고 범용적이기 때문이다.
- Queue에 Pointer 자체를 복사하면 Queue by Reference 방식도 구현할 수 있으므로 기능적으로 더 제한되지 않는다.

➤ 참고

- Queue에 복사할 데이터가 너무 크면 데이터 자체 대신 Pointer를 Queue에 복사할 수 있다.
- 함수 내부의 Stack 변수를 Queue에 직접 보낼 수 있다. Queue가 값을 복사하므로 원래 Stack 변수가 사라져도 Queue 안의 복사본은 유지한다.
- 전송 Task는 Queue에 보낸 변수나 Buffer를 즉시 다시 사용할 수 있다.
- 송신 Task와 수신 Task가 데이터 소유권과 해제 책임을 직접 맞출 필요가 줄어든다.
- RTOS가 Queue 데이터 저장용 메모리를 관리한다.
- Memory Protection이 있는 시스템에서도 값 복사를 통해 보호 경계를 넘어 데이터를 전달할 수 있다.

실시간 운영체제

□ Queue의 특성

○ Queue Write에서의 Blocking

- 가득 찬 Queue에 쓰려고 할 때도 Block Time을 지정할 수 있다.
 - 이 경우 Block Time은 Queue에 빈 공간이 생기기까지 기다리는 최대 시간이다.
 - 여러 Writer Task가 가득 찬 Queue에서 대기하고 있다면 빈 공간 하나가 생길 때 한 Task만 Unblock된다.
 - 가장 높은 Priority의 Task가 먼저 선택되고, Priority가 같다면 가장 오래 기다린 Task가 선택된다.

○ 여러 Queue에서 Blocking

- 여러 Queue를 하나의 Queue Set으로 그룹화하면 Task가 Set 안의 어느 Queue에서든 데이터가 도착할 때까지 Blocked 상태로 기다릴 수 있다

○ Queue 생성: 정적/동적 할당

- Queue는 QueueHandle_t 형의 Handle로 참조하며 사용 전에 명시적으로 생성해야 한다
 - Queue 생성 API는 xQueueCreate()와 xQueueCreateStatic()이다.
 - 각 Queue에는 Queue 자체의 Data Structure를 저장하는 RAM과, 실제 Queue 항목을 저장하는 RAM의 두 영역이 필요하다.
 - xQueueCreate()는 이 메모리를 Heap에서 동적으로 할당하고, xQueueCreateStatic()은 호출자가 미리 준비한 RAM을 사용한다.

실시간 운영체제

□ Queue의 특성

○ 여러 Task의 접근

- Queue는 독립적인 Kernel Object이며, Queue Handle을 알고 있는 어떤 Task나 ISR도 접근할 수 있다
 - 여러 Task가 하나의 Queue에 쓸 수 있고 여러 Task가 하나의 Queue에서 읽을 수도 있다.
 - 실제 설계에서는 Multiple Writer가 흔하지만 Multiple Reader는 상대적으로 덜 사용한다.

○ Queue Read에서의 Blocking

- Task가 비어 있는 Queue를 읽을 때 Block Time을 지정할 수 있다.
 - 지정한 시간 동안 데이터가 들어오기를 기다리며 Task는 Blocked 상태에 머문다.
 - 다른 Task나 Interrupt가 Queue에 데이터를 넣으면 즉시 Ready 상태로 이동한다.
 - 데이터가 오기 전에 Block Time이 만료해도 Ready 상태로 이동한다.
 - 여러 Reader Task가 같은 Queue에서 데이터를 기다릴 수 있다.
 - 데이터 하나가 들어오면 오직 하나의 Task만 Unblock된다.
 - 가장 높은 Priority의 대기 Task가 먼저 선택되고, Priority가 같다면 가장 오래 기다린 Task가 선택된다.

Using a Queue

실시간 운영체제

□ Queue 사용

○ xQueueCreate() API 함수

```
QueueHandle_t xQueueCreate( UBaseType_t uxQueueLength, UBaseType_t uxItemSize );
```

○ xQueueCreate() API 함수 프로토타입

매개변수/반환값	설명
uxQueueLength	Queue가 동시에 저장할 수 있는 최대 항목 수이다.
uxItemSize	각 데이터 항목의 크기를 바이트 단위로 지정한다.
NULL 반환	Queue 구조와 데이터 저장 영역을 할당할 Heap Memory가 부족하여 생성에 실패했다.
Non-NULL 반환	Queue 생성에 성공했으며 반환값은 생성된 Queue Handle이다.

➤ 참고

- xQueueReset()은 이미 생성된 Queue를 원래의 빈 상태로 초기화한다.

실시간 운영체제

□ Queue 사용

○ xQueueSendToBack() / xQueueSendToFront()

- xQueueSendToBack()은 Queue의 뒤쪽(Tail)에 데이터를 전송하고 xQueueSendToFront()는 앞쪽(Head)에 전송한다. xQueueSend()은 xQueueSendToBack()과 완전히 동일하다.
- ISR 주의
 - ISR에서는 xQueueSendToFront()나 xQueueSendToBack()을 호출하지 않는다.
 - 대신 xQueueSendToFrontFromISR() 또는 xQueueSendToBackFromISR()를 사용한다.

```
BaseType_t xQueueSendToFront( QueueHandle_t xQueue,  
                             const void * pvltemToQueue,  
                             TickType_t xTicksToWait );
```

```
BaseType_t xQueueSendToBack( QueueHandle_t xQueue,  
                             const void * pvltemToQueue,  
                             TickType_t xTicksToWait );
```

➤ 참고

- xQueueReset()은 이미 생성된 Queue를 원래의 빈 상태로 초기화한다.

실시간 운영체제

□ Queue 사용

○ Listing 5.2 / 5.3 Queue 앞/뒤로 전송하는 API 함수 프로토타입

항목	설명
xQueue	데이터를 보낼 Queue Handle이다.
pvItemToQueue	Queue 내부로 복사할 데이터의 주소이다. Queue 생성 시 지정한 Item Size만큼 복사한다.
xTicksToWait	Queue가 가득 찼을 때 빈 공간이 생기기를 기다릴 최대 Tick 수이다. 0이면 즉시 반환한다. pdMS_TO_TICKS()로 밀리초를 Tick으로 변환할 수 있다.
portMAX_DELAY	INCLUDE_vTaskSuspend=1인 경우 무한 대기를 의미한다.
pdPASS	Queue에 데이터를 성공적으로 썼다.
errQUEUE_FULL / pdFAIL	Queue가 가득 차 있어 데이터를 쓰지 못했다. Block Time을 지정했다면 그 시간이 만료될 때까지 공간이 생기지 않았다.

실시간 운영체제

□ Queue 사용

○ xQueueReceive() API 함수

- xQueueReceive()는 Queue에서 항목 하나를 읽으며, 읽은 항목은 Queue에서 제거한다.
- ISR 주의
 - ISR에서는 xQueueReceive() 대신 xQueueReceiveFromISR()을 사용한다.

```
BaseType_t xQueueReceive( QueueHandle_t xQueue,  
                          void * const pvBuffer,  
                          TickType_t xTicksToWait );
```

○ Listing 5.4 xQueueReceive() API 함수 프로토타입

항목	설명
xQueue	데이터를 읽을 Queue Handle이다.
pvBuffer	수신 데이터를 복사할 메모리의 주소이다. Queue 생성 시 지정한 Item Size 이상의 공간이 필요하다.
xTicksToWait	Queue가 비었을 때 데이터가 도착하기를 기다릴 최대 Tick 수이다. 0이면 즉시 반환한다.
pdPASS	데이터를 성공적으로 읽었다.
errQUEUE_EMPTY / pdFAIL	Queue가 비어 있어 데이터를 읽지 못했다. Block Time을 지정했다면 대기 시간이 만료했다.

실시간 운영체제

□ Queue 사용

○ 5.3.4 uxQueueMessagesWaiting() API 함수

- uxQueueMessagesWaiting()은 현재 Queue에 들어 있는 항목 수를 반환한다. 0을 반환하면 Queue가 비어 있다.
- ISR 주의
 - ISR에서는 uxQueueMessagesWaiting() 대신 uxQueueMessagesWaitingFromISR()을 사용한다.

```
UBaseType_t uxQueueMessagesWaiting( QueueHandle_t xQueue );
```

실시간 운영체제

□ Queue 사용

○ Example 5.1 Queue 수신에서 Blocking

- 예제 5.1은 Queue를 생성하고, 두 개의 Sender Task가 같은 Queue로 int32_t 값을 보내며, 하나의 Receiver Task가 Queue에서 값을 읽는 과정을 보여준다.
 - Sender는 Block Time을 0으로 사용하고 Receiver는 100 ms의 Block Time을 사용한다.
 - Receiver Task의 Priority가 Sender Task보다 높으므로 Sender가 값을 하나 넣는 즉시 Receiver가 Unblock 되어 Sender를 Preempt하고 데이터를 꺼낸다.
 - 따라서 Queue의 Length를 5로 생성해도 실제로는 대부분 1개를 넘지 않는다.

실시간 운영체제

□ Queue 사용

○ Example 5.1 Queue 수신에서 Blocking

```
static void vSenderTask( void *pvParameters )
{
    int32_t IValueToSend; BaseType_t xStatus;

    /* Two instances of this task are created so the value that is sent to the queue is
    passed in via the task parameter - this way each instance can use a different value.
    The queue was created to hold values of type int32_t, so cast the parameter to the
    required type. */

    IValueToSend = ( int32_t ) pvParameters;

    /* As per most tasks, this task is implemented within an infinite loop. */
    for( ;; )
    {
        /* Send the value to the queue. The first parameter is the queue to which data is
        being sent. The queue was created before the scheduler was started, so before this
        task started to execute. The second parameter is the address of the data to be sent, in
        this case the address of IValueToSend. The third parameter is the Block time – the
        time the task should be kept in the Blocked state to wait for space to become
        available on the queue should the queue already be full.*/
```

실시간 운영체제

□ Queue 사용

○ Example 5.1 Queue 수신에서 Blocking

In this case a block time is not specified because the queue should never contain more than one item, and therefore never be full. */

```
xStatus = xQueueSendToBack( xQueue, &IValueToSend, 0 );
```

```
if( xStatus != pdPASS )
```

```
{
```

```
/* The send operation could not complete because the queue was full- this must be  
an error as the queue should never contain more than one item! */
```

```
    vPrintString( "Could not send to the queue.\r\n" );
```

```
}
```

```
}
```

```
}
```

실시간 운영체제

□ Queue 사용

○ Listing 5.6 Example 5.1 Sender Task

```
static void vReceiverTask( void *pvParameters )
{
    /* Declare the variable that will hold the values received from the queue. */
    int32_t lReceivedValue;
    BaseType_t xStatus;
    const TickType_t xTicksToWait = pdMS_TO_TICKS( 100 );
    /* This task is also defined within an infinite loop. */
    for( ;; )
    {
        /* This call should always find the queue empty because this task will immediately
        remove any data that is written to the queue. */
        if( uxQueueMessagesWaiting( xQueue ) != 0 )
        {
            vPrintString( "Queue should have been empty!\n" );
        }
    }
}
```

실시간 운영체제

□ Queue 사용

○ Listing 5.6 Example 5.1 Sender Task

```
/* Receive data from the queue. The first parameter is the queue from which data is
to be received. The queue is created before the scheduler is started, and therefore this
task runs for the first time. The second parameter is the buffer into which the received
data will be placed. In this case the buffer is simply the address of a variable that has
the required size to hold the received data. The last parameter is the block time – the
maximum amount of time that the task will remain in the Blocked state to wait for
data to be available should the queue already be empty. */
```

```
    xStatus = xQueueReceive( xQueue, &lReceivedValue, xTicksToWait );
```

```
    if( xStatus == pdPASS )
```

```
    { /* Data was successfully received from the queue, print out the received value. */
```

```
        vPrintStringAndNumber( "Received = ", lReceivedValue );
```

```
    }
```

```
    else
```

```
    { /* Data was not received from the queue even after waiting for 100ms. This must
be an error as the sending tasks are free running and will be continuously writing to
the queue. */
```

실시간 운영체제

□ Queue 사용

○ Listing 5.6 Example 5.1 Sender Task

```
vPrintString( "Could not receive from the queue.\r\n" );  
    }  
}  
}
```

실시간 운영체제

□ Queue 사용

○ Listing 5.7 Example 5.1 Receiver Task

```
/* Declare a variable of type QueueHandle_t. This is used to store the
   handle to the queue that is accessed by all three tasks. */
QueueHandle_t xQueue;

int main(void)
{
    /* The queue is created to hold a maximum of 5 values, each of which is
       large enough to hold a variable of type int32_t. */
    xQueue = xQueueCreate(5, sizeof(int32_t));

    if (xQueue != NULL)
    {
```

실시간 운영체제

□ Queue 사용

○ Listing 5.7 Example 5.1 Receiver Task

```
/* Create two instances of the task that will send to the queue. The
   task parameter is used to pass the value that the task will write
   to the queue, so one task will continuously write 100 to the queue
   while the other task will continuously write 200 to the queue. Both
   tasks are created at priority 1. */
xTaskCreate(vSenderTask, "Sender1", 1000, (void *)100, 1, NULL);
xTaskCreate(vSenderTask, "Sender2", 1000, (void *)200, 1, NULL);

/* Create the task that will read from the queue. The task is created
   with priority 2, so above the priority of the sender tasks. */
xTaskCreate(vReceiverTask, "Receiver", 1000, NULL, 2, NULL);

/* Start the scheduler so the created tasks start executing. */
vTaskStartScheduler();

}
```

실시간 운영체제

□ Queue 사용

○ Listing 5.7 Example 5.1 Receiver Task

```
else
{
    /* The queue could not be created. */
}

/* If all is well then main() will never reach here as the scheduler will
   now be running the tasks. If main() does reach here then it is likely
   that there was insufficient FreeRTOS heap memory available for the idle
   task to be created. Chapter 3 provides more information on heap memory
   management. */
for (;;)
}
```

실시간 운영체제

□ Queue 사용

○ Listing 5.8 Example 5.1의 main()

➤ Figure 5.3 – Example 5.1 실행 순서

단계	상태/의미
1	Sender가 Queue에 값을 넣는다.
2	높은 Priority의 Receiver가 즉시 Ready가 되어 Sender를 Preempt한다.
3	Receiver가 값을 읽어 Queue를 다시 비운다.
4	Receiver는 다음 데이터를 기다리며 Blocked 상태가 되고 다른 Sender가 실행한다.

Receiving Data From Multiple Sources

실시간 운영체제

□ 여러 Source에서 데이터 수신

○ 여러 Source의 데이터를 하나의 Queue로 수신

- 하나의 Task가 여러 Source에서 데이터를 받는 경우, 데이터 값뿐 아니라 출처 또는 의미도 알아야 한다.
 - 데이터 값과 Source 식별자를 함께 가진 Structure를 하나의 Queue로 전송하는 방식이 간단하고 효과적이다.

○ Figure 5.4 구성 요소

구성 요소	역할
Controller Task	여러 Source의 Message를 받고 Source/Type을 확인하여 시스템 상태에 반응한다.
CAN Bus Task	CAN Message를 수신·해석한 뒤 Motor Speed 등 해석된 값을 Data_t로 전달한다.
HMI Task	사용자 명령이나 Set Point 변경을 감지하여 Data_t로 Controller에 전달한다.

실시간 운영체제

□ 여러 Source에서 데이터 수신

○ Example 5.2 Queue 송신에서 Blocking과 Structure 전송

- Example 5.1과 비슷하지만 Task Priority 관계를 반대로 설정한다.
 - Sender Task가 Receiver Task보다 높은 Priority를 가진다.
 - Queue에는 정수 대신 Data_t Structure를 저장한다.
 - Queue가 가득 차면 Sender는 최대 100 ms 동안 빈 공간을 기다리며 Blocked 상태가 된다.

실시간 운영체제

□ 여러 Source에서 데이터 수신

○ Listing 5.9 Queue로 전달하는 Structure 정의

```
/* Define an enumerated type used to identify the source of the data. */
typedef enum
{
    eSender1,
    eSender2
} DataSource_t;

/* Define the structure type that will be passed on the queue. */
typedef struct
{
    uint8_t ucValue;
    DataSource_t eDataSource;
} Data_t;

/* Declare two variables of type Data_t that will be passed on the queue. */
static const Data_t xStructsToSend[ 2 ] =
{
    { 100, eSender1 }, /* Used by Sender1. */
    { 200, eSender2 } /* Used by Sender2. */
};
```

실시간 운영체제

□ 여러 Source에서 데이터 수신

○ Listing 5.10 Example 5.2 Sender Task

```
static void vSenderTask( void *pvParameters )
{
    BaseType_t xStatus;
    const TickType_t xTicksToWait = pdMS_TO_TICKS( 100 );

    /* As per most tasks, this task is implemented within an infinite loop. */
    for( ;; )
    {
        /* Send to the queue.
        The second parameter is the address of the structure being sent. The
        address is passed in as the task parameter so pvParameters is used
        directly.
        The third parameter is the Block time - the time the task should be
        kept in the Blocked state to wait for space to become available on
        the queue if the queue is already full. */
```

실시간 운영체제

□ 여러 Source에서 데이터 수신

○ Listing 5.10 Example 5.2 Sender Task

```
/* A block time is specified because the sending tasks have a higher
priority than the receiving task so the queue is expected to become
full. The receiving task will remove items from the queue when both
sending tasks are in the Blocked state. */
xStatus = xQueueSendToBack( xQueue, pvParameters, xTicksToWait );

if( xStatus != pdPASS )
{
    /* The send operation could not complete, even after waiting for
    100ms. This must be an error as the receiving task should make
    space in the queue as soon as both sending tasks are in the
    Blocked state. */
    vPrintString( "Could not send to the queue.WrWn" );
}
}
```

실시간 운영체제

□ 여러 Source에서 데이터 수신

○ Listing 5.11 Example 5.2 Receiver Task

```
static void vReceiverTask( void *pvParameters )
{
    /* Declare the structure that will hold the values received from the
    queue. */
    Data_t xReceivedStructure;
    BaseType_t xStatus;

    /* This task is also defined within an infinite loop. */
    for( ;; )
    {
        /* Because it has the lowest priority this task will only run when the
        sending tasks are in the Blocked state. The sending tasks will only
        enter the Blocked state when the queue is full so this task always
        expects the number of items in the queue to be equal to the queue
        length, which is 3 in this case. */
        if( uxQueueMessagesWaiting( xQueue ) != 3 )
        {
            vPrintString( "Queue should have been full!\r\n" );
        }
    }
}
```

실시간 운영체제

□ 여러 Source에서 데이터 수신

○ Listing 5.11 Example 5.2 Receiver Task

```
/* Receive from the queue.  
The second parameter is the buffer into which the received data will  
be placed. In this case the buffer is simply the address of a  
variable that has the required size to hold the received structure.  
The last parameter is the block time - the maximum amount of time  
that the task will remain in the Blocked state to wait for data to  
be available if the queue is already empty. In this case a block  
time is not necessary because this task will only run when the  
queue is full. */  
xStatus = xQueueReceive( xQueue, &xReceivedStructure, 0 );  
  
if( xStatus == pdPASS )  
{  
    /* Data was successfully received from the queue, print out the  
    received value and the source of the value. */
```

실시간 운영체제

□ 여러 Source에서 데이터 수신

○ Listing 5.11 Example 5.2 Receiver Task

```
    if( xReceivedStructure.eDataSource == eSender1 )
    {
        vPrintStringAndNumber( "From Sender 1 = ",
                               xReceivedStructure.ucValue );
    }
    else
    {
        vPrintStringAndNumber( "From Sender 2 = ",
                               xReceivedStructure.ucValue );
    }
}
else
{
    /* Nothing was received from the queue. This must be an error as
    this task should only run when the queue is full. */
    vPrintString( "Could not receive from the queue. Wrrn" );
}
}
```

실시간 운영체제

- 여러 Source에서 데이터 수신
 - Listing 5.12 Example 5.2의 main()

```
int main( void )
{
    /* The queue is created to hold a maximum of 3 structures of type Data_t. */
    xQueue = xQueueCreate( 3, sizeof( Data_t ) );

    if( xQueue != NULL )
    {
        /* Create two instances of the task that will write to the queue. The
        parameter is used to pass the structure that the task will write to
        the queue, so one task will continuously send xStructsToSend[ 0 ]
        to the queue while the other task will continuously send
        xStructsToSend[ 1 ]. Both tasks are created at priority 2, which is
        above the priority of the receiver. */
        xTaskCreate( vSenderTask, "Sender1", 1000, &( xStructsToSend[ 0 ] ),
                    2, NULL );
        xTaskCreate( vSenderTask, "Sender2", 1000, &( xStructsToSend[ 1 ] ),
                    2, NULL );
    }
}
```

실시간 운영체제

- 여러 Source에서 데이터 수신
 - Listing 5.12 Example 5.2의 main()

```
/* Create the task that will read from the queue. The task is created
with priority 1, so below the priority of the sender tasks. */
xTaskCreate( vReceiverTask, "Receiver", 1000, NULL, 1, NULL );

/* Start the scheduler so the created tasks start executing. */
vTaskStartScheduler();
}
else
{
    /* The queue could not be created. */
}

/* If all is well then main() will never reach here as the scheduler will
now be running the tasks. If main() does reach here then it is likely
that there was insufficient heap memory available for the idle task to
be created. Chapter 3 provides more information on heap memory
management. */
for( ;; );
}
```

실시간 운영체제

□ 여러 Source에서 데이터 수신

○ Figure 5.6 Sender Priority가 더 높은 경우의 실행 흐름

단계	상태/의미
t1	Sender 1이 Queue를 3개 항목으로 채운다.
t2	Queue가 가득 차 Sender 1이 Blocked 상태가 되고 Sender 2가 실행한다.
t3	Sender 2도 Queue가 가득 차 Blocked 상태가 되고 Receiver가 실행한다.
t4	Receiver가 한 항목을 제거하면 오래 기다린 Sender 1이 먼저 Unblock되어 Receiver를 Preempt한다.

실시간 운영체제

□ 여러 Source에서 데이터 수신

○ Figure 5.6 Sender Priority가 더 높은 경우의 실행 흐름

단계	상태/의미
t5	Sender 1이 빈 자리를 채우고 다시 Blocked 상태로 들어간다.
t6	Receiver가 다시 한 항목을 제거하면 이번에는 더 오래 기다린 Sender 2가 실행한다.
t7	Sender 2가 항목을 넣고 Blocked 상태로 돌아가며 Receiver가 다시 실행할 수 있게 된다.

Working with Large or Variable Sized Data

실시간 운영체제

□ 크거나 가변 길이 데이터 처리

○ Pointer를 Queue에 저장

- 데이터가 크면 데이터 자체를 복사하기보다 Pointer를 Queue에 전송하는 편이 처리 시간과 RAM 사용량 측면에서 효율적이다.
 - Pointer가 가리키는 RAM의 Owner를 명확히 정의한다.
 - Queue에 보내기 전에는 Sender만, 수신 후에는 Receiver만 해당 메모리에 접근하도록 설계하는 것이 이상적이다.
 - 동적으로 할당한 메모리는 정확히 하나의 Task만 해제 책임을 가진다.
 - 메모리를 해제한 뒤에는 어떤 Task도 다시 접근하면 안 된다.
 - Task Stack에 할당한 데이터의 주소는 다른 Task로 전달하지 않는다.

실시간 운영체제

□ 크거나 가변 길이 데이터 처리

○ Listing 5.13 Pointer를 저장하는 Queue 생성

```
/* Declare a variable of type QueueHandle_t to hold the handle of the
   queue being created. */
QueueHandle_t xPointerQueue;

/* Create a queue that can hold a maximum of 5 pointers, in this case
   character pointers. */
xPointerQueue = xQueueCreate( 5, sizeof( char * ) );
```

실시간 운영체제

□ 크거나 가변 길이 데이터 처리

○ Listing 5.14 Buffer Pointer 전송

```
/* A task that obtains a buffer, writes a string to the buffer, then
   sends the address of the buffer to the queue created in Listing 5.13. */
void vStringSendingTask( void *pvParameters )
{
    char *pcStringToSend;
    const size_t xMaxStringLength = 50;
    BaseType_t xStringNumber = 0;

    for( ;; )
    {
        /* Obtain a buffer that is at least xMaxStringLength characters big.
           The implementation of prvGetBuffer() is not shown – it might obtain
           the buffer from a pool of pre-allocated buffers, or just allocate
           the buffer dynamically. */
        pcStringToSend = ( char * ) prvGetBuffer( xMaxStringLength );
```

실시간 운영체제

□ 크거나 가변 길이 데이터 처리

○ Listing 5.14 Buffer Pointer 전송

```
/* Write a string into the buffer. */
snprintf( pcStringToSend, xMaxStringLength, "String number %d\r\n",
          xStringNumber );

/* Increment the counter so the string is different on each iteration
of this task. */
xStringNumber++;

/* Send the address of the buffer to the queue that was created in
Listing 5.13. The address of the buffer is stored in the
pcStringToSend variable.*/
xQueueSend( xPointerQueue, /* The handle of the queue. */
            &pcStringToSend, /* The address of the pointer that points
                               to the buffer. */
            portMAX_DELAY );
    }
}
```

실시간 운영체제

□ 크거나 가변 길이 데이터 처리

○ Listing 5.15 Buffer Pointer 수신 및 해제

```
/* A task that receives the address of a buffer from the queue created
   in Listing 5.13, and written to in Listing 5.14. The buffer contains a
   string, which is printed out. */
void vStringReceivingTask( void *pvParameters )
{
    char *pcReceivedString;

    for( ;; )
    {
        /* Receive the address of a buffer. */
        xQueueReceive( xPointerQueue, /* The handle of the queue. */
                      &pcReceivedString, /* Store the buffer's address in
                                           pcReceivedString. */
                      portMAX_DELAY );

        /* The buffer holds a string, print it out. */
        vPrintString( pcReceivedString );

        /* The buffer is not required any more - release it so it can be freed,
           or re-used. */
        prvReleaseBuffer( pcReceivedString );
    }
}
```

실시간 운영체제

□ 크거나 가변 길이 데이터 처리

○ 서로 다른 Type과 Length의 데이터 전송

- Structure 전송과 Pointer 전송을 결합하면 하나의 Queue를 통해 서로 다른 Source의 서로 다른 Type/Length 데이터를 받을 수 있다.
 - FreeRTOS+TCP의 TCP/IP Task가 대표적인 실전 예이다.
 - IPStackEvent_t의 eEventType은 Event 종류를 식별하고 pvData는 값 또는 Buffer를 가리키는 Generic Pointer로 사용한다.

Event	eEventType	pvData 의미
Network RX	eNetworkRxEvent	수신된 Network Buffer를 가리킨다.
TCP Accept	eTCPAcceptEvent	Connection을 Accept하는 Socket Handle을 담는다.
Network Down	eNetworkDownEvent	데이터가 없어 NULL을 사용한다.

실시간 운영체제

□ 크거나 가변 길이 데이터 처리

○ Listing 5.16 FreeRTOS+TCP Event Structure

```
/* A subset of the enumerated types used in the TCP/IP stack to
   identify events. */
typedef enum
{
    eNetworkDownEvent = 0, /* The network interface has been lost, or needs
                           (re)connecting. */
    eNetworkRxEvent,      /* A packet has been received from the network. */
    eTCPAcceptEvent,      /* FreeRTOS_accept() called to accept or wait for a
                           new client. */
    /* Other event types appear here but are not shown in this listing. */
} eIPEvent_t;

/* The structure that describes events, and is sent on a queue to the
   TCP/IP task. */
typedef struct IP_TASK_COMMANDS
{
    /* An enumerated type that identifies the event. */
    eIPEvent_t eEventType;

    /* A generic pointer that can hold a value, or point to a buffer. */
    void *pvData;
} IPStackEvent_t;
```

실시간 운영체제

□ 크거나 가변 길이 데이터 처리

○ Listing 5.17 Network Data 전달

```
void vSendRxDataToTheTCPTask( NetworkBufferDescriptor_t *pRxedData )
{
    IPStackEvent_t xEventStruct;

    /* Complete the IPStackEvent_t structure. The received data is stored in
    pRxedData. */
    xEventStruct.eEventType = eNetworkRxEvent;
    xEventStruct.pvData = ( void * ) pRxedData;

    /* Send the IPStackEvent_t structure to the TCP/IP task. */
    xSendEventStructToIPTask( &xEventStruct );
}
```

실시간 운영체제

- 크거나 가변 길이 데이터 처리
 - Listing 5.18 Socket Handle 전달

```
void vSendAcceptRequestToTheTCPTask( Socket_t xSocket )
{
    IPStackEvent_t xEventStruct;

    /* Complete the IPStackEvent_t structure. */
    xEventStruct.eEventType = eTCPAcceptEvent;
    xEventStruct.pvData = ( void * ) xSocket;

    /* Send the IPStackEvent_t structure to the TCP/IP task. */
    xSendEventStructToIPTask( &xEventStruct );
}
```

실시간 운영체제

□ 크거나 가변 길이 데이터 처리

○ Listing 5.19 Network Down Event 전달

```
void vSendNetworkDownEventToTheTCPTask( Socket_t xSocket )
{
    IPStackEvent_t xEventStruct;

    /* Complete the IPStackEvent_t structure. */
    xEventStruct.eEventType = eNetworkDownEvent;
    xEventStruct.pvData = NULL; /* Not used, but set to NULL for completeness. */

    /* Send the IPStackEvent_t structure to the TCP/IP task. */
    xSendEventStructToIPTask( &xEventStruct );
}
```

실시간 운영체제

- 크거나 가변 길이 데이터 처리
 - Listing 5.20 TCP/IP Task Event 처리

```
IPStackEvent_t xReceivedEvent;

/* Block on the network event queue until either an event is received, or
   xNextIPSleep ticks pass without an event being received. eEventType is
   set to eNoEvent in case the call to xQueueReceive() returns because it
   timed out, rather than because an event was received. */
xReceivedEvent.eEventType = eNoEvent;
xQueueReceive( xNetworkEventQueue, &xReceivedEvent, xNextIPSleep );

/* Which event was received, if any? */
switch( xReceivedEvent.eEventType )
{
    case eNetworkDownEvent :
        /* Attempt to (re)establish a connection. This event is not
           associated with any data. */
        prvProcessNetworkDownEvent();
        break;
```

실시간 운영체제

□ 크거나 가변 길이 데이터 처리

○ Listing 5.20 TCP/IP Task Event 처리

```
case eNetworkRxEvent:
    /* The network interface has received a new packet. A pointer to the
       received data is stored in the pvData member of the received
       IPStackEvent_t structure. Process the received data. */
    prvHandleEthernetPacket( ( NetworkBufferDescriptor_t * )
                             ( xReceivedEvent.pvData ) );

    break;

case eTCPAcceptEvent:
    /* The FreeRTOS_accept() API function was called. The handle of the
       socket that is accepting a connection is stored in the pvData
       member of the received IPStackEvent_t structure. */
    xSocket = ( FreeRTOS_Socket_t * ) ( xReceivedEvent.pvData );
    xTCPCheckNewClient( xSocket );
    break;

/* Other event types are processed in the same way, but are not shown
here. */
}
```

Receiving From Multiple Queues

실시간 운영체제

□ 여러 Queue에서 수신

○ Queue Set

- 하나의 Task가 서로 다른 크기·의미·Source의 데이터를 받아야 할 때는 단일 Structure Queue가 가장 간결한 설계이다.
 - Third-Party Code 등 설계 제약으로 Source별 전용 Queue를 유지해야 할 때 Queue Set을 사용할 수 있다.
 - Queue Set을 사용하면 여러 Queue를 반복 Polling하지 않고 Set 내부 어느 Queue에서든 데이터가 도착할 때까지 Blocked 상태로 기다릴 수 있다.
- 설계 권고
 - Queue Set은 단일 Structure Queue보다 덜 간결하고 비효율적이므로 반드시 필요한 경우에만 사용하는 것이 권장된다.
 - Queue Set 기능을 사용하려면 FreeRTOSConfig.h에서 configUSE_QUEUE_SETS를 1로 설정한다.

실시간 운영체제

□ 여러 Queue에서 수신

○ xQueueCreateSet() API 함수

- Queue Set은 QueueSetHandle_t Handle로 참조하며 사용 전에 명시적으로 생성한다.

```
QueueSetHandle_t xQueueCreateSet( const UBaseType_t uxEventQueueLength );
```

매개변수/반환값	설명
uxEventQueueLength	Queue Set이 동시에 보관할 수 있는 Queue/Semaphore Handle의 최대 개수이다.
NULL 반환	Heap Memory 부족으로 Queue Set을 생성하지 못했다.
Non-NULL 반환	Queue Set 생성에 성공했으며 반환값은 Queue Set Handle이다.

실시간 운영체제

□ 여러 Queue에서 수신

○ xQueueAddToSet() API 함수

- Queue 또는 Semaphore를 Queue Set에 추가한다.

```
BaseType_t xQueueAddToSet( QueueSetMemberHandle_t xQueueOrSemaphore,  
                           QueueSetHandle_t xQueueSet );
```

항목	설명
xQueueOrSemaphore	Queue Set에 추가할 Queue 또는 Semaphore Handle이다.
xQueueSet	대상 Queue Set Handle이다.
pdPASS	멤버 추가에 성공했다.
pdFAIL	Queue 또는 Semaphore를 Set에 추가할 수 없었다.

실시간 운영체제

□ 여러 Queue에서 수신

- xQueueSelectFromSet() API 함수

- Queue Set에서 데이터가 준비된 멤버의 Handle을 읽는다.
 - Handle이 반환되면 해당 Queue 또는 Semaphore에는 사용할 데이터/자원이 있다는 뜻이다.

[illegible]

실시간 운영체제

□ 여러 Queue에서 수신

○ Listing 5.23 xQueueSelectFromSet() 매개변수 / 반환값

항목	설명
xQueueSet	읽을 Queue Set Handle이다.
xTicksToWait	모든 멤버가 비었을 때 Handle이 도착하기를 기다릴 최대 Tick 수이다.
Non-NULL 반환	데이터를 가진 Queue 또는 Semaphore의 Handle이다.
NULL 반환	Block Time 동안 어떤 멤버에서도 Event가 발생하지 않아 Timeout이 발생했다.

실시간 운영체제

□ 여러 Queue에서 수신

○ Example 5.3 Queue Set 사용

- Sender Task 두 개가 서로 다른 Queue에 데이터를 보내고, Receiver Task 하나가 두 Queue가 포함된 Queue Set을 기다린다.
 - 각 Queue는 char * 한 개를 저장한다.
 - Receiver의 Priority가 Sender보다 높다.
 - Sender 1은 100 ms, Sender 2는 200 ms 주기로 데이터를 전송한다.

실시간 운영체제

□ 여러 Queue에서 수신

○ Listing 5.24 Example 5.3의 main()

```
/* Declare two variables of type QueueHandle_t. Both queues are added
   to the same queue set. */
static QueueHandle_t xQueue1 = NULL, xQueue2 = NULL;

/* Declare a variable of type QueueSetHandle_t. This is the queue set
   to which the two queues are added. */
static QueueSetHandle_t xQueueSet = NULL;

int main( void )
{
    /* Create the two queues, both of which send character pointers. The
       priority of the receiving task is above the priority of the sending
       tasks, so the queues will never have more than one item in them at
       any one time*/
    xQueue1 = xQueueCreate( 1, sizeof( char * ) );
    xQueue2 = xQueueCreate( 1, sizeof( char * ) );
```

실시간 운영체제

□ 여러 Queue에서 수신

○ Listing 5.24 Example 5.3의 main()

```
/* Create the queue set. Two queues will be added to the set, each of
which can contain 1 item, so the maximum number of queue handles the
queue set will ever have to hold at one time is 2. */
xQueueSet = xQueueCreateSet( 1 * 2 );

/* Add the two queues to the set. */
xQueueAddToSet( xQueue1, xQueueSet );
xQueueAddToSet( xQueue2, xQueueSet );

/* Create the tasks that send to the queues. */
xTaskCreate( vSenderTask1, "Sender1", 1000, NULL, 1, NULL );
xTaskCreate( vSenderTask2, "Sender2", 1000, NULL, 1, NULL );

/* Create the task that reads from the queue set to determine which of
the two queues contain data. */
xTaskCreate( vReceiverTask, "Receiver", 1000, NULL, 2, NULL );

/* Start the scheduler so the created tasks start executing. */
vTaskStartScheduler();

for( ;; );
return 0;
}
```

실시간 운영체제

□ 여러 Queue에서 수신

○ Listing 5.25 Example 5.3 Sender Task

```
void vSenderTask1( void *pvParameters )
{
    const TickType_t xBlockTime = pdMS_TO_TICKS( 100 );
    const char * const pcMessage = "Message from vSenderTask1\r\n";

    /* As per most tasks, this task is implemented within an infinite loop. */
    for( ;; )
    {
        /* Block for 100ms. */
        vTaskDelay( xBlockTime );

        /* Send this task's string to xQueue1. It is not necessary to use a
        block time, even though the queue can only hold one item. This is
        because the priority of the task that reads from the queue is
        higher than the priority of this task. */
        xQueueSend( xQueue1, &pcMessage, 0 );
    }
}
```

실시간 운영체제

□ 여러 Queue에서 수신

○ Listing 5.25 Example 5.3 Sender Task

```
void vSenderTask2( void *pvParameters )
{
    const TickType_t xBlockTime = pdMS_TO_TICKS( 200 );
    const char * const pcMessage = "Message from vSenderTask2\r\n";

    /* As per most tasks, this task is implemented within an infinite loop. */
    for( ;; )
    {
        /* Block for 200ms. */
        vTaskDelay( xBlockTime );

        /* Send this task's string to xQueue2. It is not necessary to use a
        block time, even though the queue can only hold one item. This is
        because the priority of the task that reads from the queue is
        higher than the priority of this task. */
        xQueueSend( xQueue2, &pcMessage, 0 );
    }
}
```

실시간 운영체제

□ 여러 Queue에서 수신

○ Listing 5.26 Example 5.3 Receiver Task

```
void vReceiverTask( void *pvParameters )
{
    QueueHandle_t xQueueThatContainsData;
    char *pcReceivedString;

    /* As per most tasks, this task is implemented within an infinite loop. */
    for( ;; )
    {
        /* Block on the queue set to wait for one of the queues in the set to
        contain data. Cast the QueueSetMemberHandle_t value returned from
        xQueueSelectFromSet() to a QueueHandle_t, as it is known all the
        members of the set are queues. */
        xQueueThatContainsData = ( QueueHandle_t ) xQueueSelectFromSet(
            xQueueSet, portMAX_DELAY );
```

실시간 운영체제

□ 여러 Queue에서 수신

○ Listing 5.26 Example 5.3 Receiver Task

```
/* An indefinite block time was used when reading from the queue set,  
so xQueueSelectFromSet() will not have returned unless one of the  
queues in the set contained data, and xQueueThatContainsData cannot  
be NULL. Read from the queue. It is not necessary to specify a  
block time because it is known the queue contains data. */  
xQueueReceive( xQueueThatContainsData, &pcReceivedString, 0 );  
  
/* Print the string received from the queue. */  
vPrintString( pcReceivedString );  
}  
}
```

실시간 운영체제

□ 여러 Queue에서 수신

○ Example 5.3 실행 결과

- xQueueSelectFromSet()은 portMAX_DELAY로 호출되므로 Receiver는 어느 Queue에 데이터가 도착할 때까지 Blocked 상태로 기다린다.
 - 반환된 Handle이 가리키는 Queue에는 반드시 데이터가 있으므로 xQueueReceive()의 Block Time은 0을 사용해도 된다.
 - Sender 1은 100 ms마다, Sender 2는 200 ms마다 전송하므로 Sender 1의 Message가 약 두 배 자주 출력된다.

○ 더 현실적인 Queue Set 사용

- Queue Set은 서로 다른 Type의 Queue와 Semaphore를 함께 포함할 수 있다.
 - xQueueSelectFromSet()의 반환 Handle을 각 멤버 Handle과 비교하여 어떤 Object가 Ready인지 식별한다.

실시간 운영체제

□ 여러 Queue에서 수신

○ Listing 5.27 Queue와 Semaphore가 포함된 Queue Set

```
/* The handle of the queue from which character pointers are received. */
QueueHandle_t xCharPointerQueue;

/* The handle of the queue from which uint32_t values are received. */
QueueHandle_t xUint32tQueue;

/* The handle of the binary semaphore. */
SemaphoreHandle_t xBinarySemaphore;

/* The queue set to which the two queues and the binary semaphore belong. */
QueueSetHandle_t xQueueSet;

void vAMoreRealisticReceiverTask( void *pvParameters )
{
    QueueSetMemberHandle_t xHandle;
    char *pcReceivedString;
    uint32_t ulRecievedValue;
    const TickType_t xDelay100ms = pdMS_TO_TICKS( 100 );
```

실시간 운영체제

□ 여러 Queue에서 수신

○ Listing 5.27 Queue와 Semaphore가 포함된 Queue Set

```
for( ;; )
{
    /* Block on the queue set for a maximum of 100ms to wait for one of the
    members of the set to contain data. */
    xHandle = xQueueSelectFromSet( xQueueSet, xDelay100ms );

    /* Test the value returned from xQueueSelectFromSet(). */
    if( xHandle == NULL )
    {
        /* The call to xQueueSelectFromSet() timed out. */
    }
    else if( xHandle == ( QueueSetMemberHandle_t ) xCharPointerQueue )
    {
        /* The queue is known to contain data, so a block time of 0 is used. */
        xQueueReceive( xCharPointerQueue, &pcReceivedString, 0 );
        /* The received character pointer can be processed here... */
    }
}
```

실시간 운영체제

□ 여러 Queue에서 수신

○ Listing 5.27 Queue와 Semaphore가 포함된 Queue Set

```
else if( xHandle == ( QueueSetMemberHandle_t ) xUint32tQueue )
{
    /* The queue is known to contain data, so a block time of 0 is used. */
    xQueueReceive( xUint32tQueue, &ulRecievedValue, 0 );
    /* The received value can be processed here... */
}
else if( xHandle == ( QueueSetMemberHandle_t ) xBinarySemaphore )
{
    /* The semaphore is known to be available so a block time of 0 is used. */
    xSemaphoreTake( xBinarySemaphore, 0 );
    /* Whatever processing is necessary when the semaphore is taken
    can be performed here... */
}
}
```

Using a Queue to Create a Mailbox

실시간 운영체제

□ Queue를 Mailbox로 사용

○ Mailbox

- 이 책에서는 Length가 1인 Queue를 Mailbox라고 부른다.
 - 기능 자체가 Queue와 다른 것이 아니라 애플리케이션에서 사용하는 방식이 다르다.

구분	Queue 사용 방식	Mailbox 사용 방식
데이터 흐름	Sender가 넣고 Receiver가 꺼내며 데이터가 Queue를 통과한다.	최신 값을 하나 저장해 여러 Task/ISR가 읽을 수 있게 한다.
읽기 후 데이터	xQueueReceive()를 사용하면 읽은 항목이 제거된다.	xQueuePeek()을 사용하면 읽어도 항목이 남는다.
새 값 쓰기	공간이 있어야 일반적으로 Send할 수 있다.	xQueueOverwrite()로 기존 값을 최신 값으로 교체한다.

실시간 운영체제

□ Queue를 Mailbox로 사용

○ Listing 5.28 Mailbox로 사용할 Length 1 Queue 생성

```
/* A mailbox can hold a fixed size data item. The size of the data item is set
   when the mailbox (queue) is created. In this example the mailbox is created
   to hold an Example_t structure. */
typedef struct xExampleStructure
{
    TickType_t xTimeStamp;
    uint32_t ulValue;
} Example_t;

/* A mailbox is a queue, so its handle is stored in a variable of type
   QueueHandle_t. */
QueueHandle_t xMailbox;

void vAFunction( void )
{
    /* Create the queue that is going to be used as a mailbox. The queue has
       a length of 1 to allow it to be used with xQueueOverwrite(). */
    xMailbox = xQueueCreate( 1, sizeof( Example_t ) );
}
```

실시간 운영체제

□ Queue를 Mailbox로 사용

○ xQueueOverwrite() API 함수

- Queue가 이미 가득 차 있으면 기존 데이터를 덮어쓴다.
 - Length가 1인 Queue에서만 사용해야 한다.
 - ISR에서는 xQueueOverwriteFromISR()을 사용한다.

```
BaseType_t xQueueOverwrite( QueueHandle_t xQueue, const void * pvItemToQueue );
```

항목	설명
xQueue	데이터를 쓸 Queue Handle이다.
pvItemToQueue	Queue 내부로 복사할 데이터의 주소이다.
pdPASS	Queue가 가득 차도 값을 쓰므로 가능한 반환값은 pdPASS뿐이다.

실시간 운영체제

□ Queue를 Mailbox로 사용

○ Listing 5.30 Mailbox의 값을 최신 값으로 갱신

```
void vUpdateMailbox( uint32_t ulNewValue )
{
    /* Example_t was defined in Listing 5.28. */
    Example_t xData;

    /* Write the new data into the Example_t structure.*/
    xData.ulValue = ulNewValue;

    /* Use the RTOS tick count as the time stamp stored in the Example_t
    structure. */
    xData.xTimeStamp = xTaskGetTickCount();

    /* Send the structure to the mailbox - overwriting any data that is
    already in the mailbox. */
    xQueueOverwrite( xMailbox, &xData );
}
```

실시간 운영체제

□ Queue를 Mailbox로 사용

○ xQueuePeek() API 함수

- Queue의 Head에 있는 항목을 읽지만 Queue에서 제거하지 않는다.
 - Mailbox의 최신 값을 여러 Task가 반복해서 읽을 수 있다.
 - ISR에서는 xQueuePeekFromISR()을 사용한다.

```
BaseType_t xQueuePeek( QueueHandle_t xQueue,  
                      void * const pvBuffer,  
                      TickType_t xTicksToWait );
```

실시간 운영체제

□ Queue를 Mailbox로 사용

○ Listing 5.32 xQueuePeek()으로 Mailbox 읽기

```
BaseType_t vReadMailbox( Example_t *pxData )
{
    TickType_t xPreviousTimeStamp;
    BaseType_t xDataUpdated;

    /* This function updates an Example_t structure with the latest value
    received from the mailbox. Record the time stamp already contained in
    *pxData before it gets overwritten by the new data. */
    xPreviousTimeStamp = pxData->xTimeStamp;

    /* Update the Example_t structure pointed to by pxData with the data
    contained in the mailbox. If xQueueReceive() was used here then the
    mailbox would be left empty, and the data could not then be read by
    any other tasks. */
```

실시간 운영체제

□ Queue를 Mailbox로 사용

○ Listing 5.32 xQueuePeek()으로 Mailbox 읽기

```
/* Using xQueuePeek() instead of xQueueReceive() ensures the data remains
in the mailbox. An infinite block time is used, so it is not necessary
to check the value returned from xQueuePeek(). */
xQueuePeek( xMailbox, pxData, portMAX_DELAY );

/* Return pdTRUE if the value read from the mailbox has been updated since
this function was last called. Otherwise return pdFALSE. */
if( pxData->xTimeStamp > xPreviousTimeStamp )
{
    xDataUpdated = pdTRUE;
}
else
{
    xDataUpdated = pdFALSE;
}

return xDataUpdated;
}
```

실시간 운영체제

□ 강의용 핵심 정리

- Queue는 Task 간 데이터를 안전하게 전달하는 대표적인 FreeRTOS Kernel Object이다.
- Queue의 Length와 Item Size는 생성할 때 고정한다. 일반적으로 FIFO로 동작한다.
- FreeRTOS Queue는 Queue by Copy 방식이 기본이며, 큰 데이터는 Pointer를 Queue에 복사하는 방식으로 효율적으로 전달할 수 있다.
- Queue가 비었을 때 Receive, Queue가 가득 찼을 때 Send에서 Block Time을 지정할 수 있다.
- 여러 Task가 같은 Queue에서 기다리면 가장 높은 Priority가 먼저 Unblock되고, Priority가 같으면 가장 오래 기다린 Task가 선택된다.
- 여러 Source의 데이터를 받을 때는 Source/Type 정보가 포함된 Structure를 하나의 Queue로 보내는 설계가 권장된다.
- Queue Set은 여러 Queue/Semaphore 중 어느 Object에 Event가 발생했는지 Block 상태에서 기다릴 수 있게 한다.
- Mailbox는 Length 1 Queue를 xQueueOverwrite()와 xQueuePeek()으로 사용하여 최신 값을 유지하는 패턴이다.